BCSE498J Project-II / CBS1904 / CSE1904 - Capstone Project

CRACK DETECTION IN INFRASTRUCTURE USING OPTIMIZED YOLO MODELS

21BCE0720 GAURANG PANPALIA

21BCT0005 RUDRAKSH AGARWAL

21BKT0016 NITISH RAI

Under the Supervision of

Dr. ANINDITA KUNDU

Associate Professor Grade 1

School of Computer Science and Engineering (SCOPE)

B.Tech.

in

Computer Science and Engineering

School of Computer Science and Engineering

February 2025

ABSTRACT

Structural Health Monitoring (SHM) is essential for maintaining the safety, reliability, and longevity of buildings and bridges. Surface crack detection is a critical aspect of preventive maintenance, especially for large-scale infrastructures and historical structures, where undetected cracks can escalate into catastrophic failures. The 2018 collapse of the Morandi Bridge in Genoa, Italy—which tragically claimed 43 lives—highlights the devastating consequences of neglected structural damage. Similarly, the 2021 Champlain Towers South condominium collapse in Surfside, Florida, resulted in 98 fatalities due to unnoticed structural deterioration. In India, the 2016 Kolkata flyover collapse and the 2019 Mumbai footbridge collapse also underscore the severe risks associated with inadequate infrastructure monitoring. Traditional manual inspection methods are time-consuming, labour-intensive, and prone to human error, emphasizing the need for efficient, automated crack detection systems that can provide timely and accurate assessments.

This project aims to develop an optimal automated crack detection system using the advanced You Only Look Once (YOLO) family of models—YOLOv8, YOLOv9, YOLOv10, and the latest YOLOv11. These models are widely recognized for their high-speed, real-time object detection capabilities, making them ideal for rapid damage assessment. The models will be trained on diverse, high-resolution datasets that include various crack patterns under real-world conditions, such as varying lighting, occlusions, complex backgrounds, and different crack sizes and orientations. Transfer learning will further enhance the models to adapt to practical deployment scenarios.

To ensure accurate comparisons, two powerful optimization algorithms—Stochastic Gradient Descent (SGD) and AdamW—will be implemented and evaluated across all YOLO versions. SGD is efficient in handling large-scale data and is widely used in deep learning models for its simplicity and stability. AdamW, on the other hand, improves model convergence and generalization by decoupling weight decay from gradient updates. The models will be assessed based on detection accuracy, inference speed, and mean average precision (mAP) to determine the optimal configuration that balances precision and efficiency for real-time applications.

The optimized YOLO model can also be integrated with Unmanned Aerial Vehicles (UAVs) for rapid and remote crack detection in inaccessible areas. This system enhances structural health monitoring by providing a scalable and effective solution for real-time crack detection in buildings and bridges. By advancing automated inspection technologies, this project contributes to proactive maintenance, minimizing the risk of catastrophic failures, and ensuring the long-term resilience of critical infrastructure.

Keywords - Machine Learning, Neural Networks, Anomaly Detection, YOLO, ADAMW, SGD.

TABLE OF CONTENTS

1. INTRODUCTION

1.1 Background

Infrastructure such as bridges, roads, buildings, and tunnels forms the backbone of modern civilization, enabling economic growth, transportation, and societal development. However, over time, these structures deteriorate due to aging, environmental conditions, mechanical stress, and external factors such as seismic activity, temperature fluctuations, and corrosion. Cracks are among the earliest indicators of structural failure, and timely detection is crucial to prevent catastrophic failures.

Traditional crack detection techniques, such as visual inspection, ultrasonic testing, and infrared thermography, require extensive manpower, time, and specialized equipment. Moreover, manual inspections are highly subjective and error-prone, often leading to inconsistent and delayed assessments. Recent advancements in computer vision and deep learning have significantly transformed crack detection by enabling real-time, automated, and highly accurate assessments.

Deep learning models, particularly the YOLO (You Only Look Once) family, have proven to be highly effective for crack detection due to their speed and accuracy. YOLO is a single-stage object detection model that processes images in real time, making it suitable for large-scale infrastructure monitoring. With successive improvements in architecture, from YOLOv4 to YOLOv11, these models have been optimized to detect even the smallest cracks under complex lighting, occlusions, and varying surface conditions.

This research focuses on leveraging the latest YOLO models (YOLOv8–YOLOv11) to develop an optimized, real-time crack detection system. By evaluating different versions and optimizing training methods, the study aims to improve crack detection accuracy and efficiency, making structural health monitoring more effective and scalable.

1.2 Motivation

The need for a reliable, automated crack detection system is reinforced by real-world infrastructure failures, where undetected structural damage led to disastrous consequences:

- Morandi Bridge Collapse (Italy, 2018) – Caused by structural weakening, this disaster resulted in 43 deaths and emphasized the risks of delayed inspections.
- Champlain Towers South Collapse (USA, 2021) – A condominium collapsed unexpectedly, killing 98 people, due to long-term concrete deterioration and ignored warnings.
- Kolkata Flyover Collapse (India, 2016) – A partially constructed flyover collapsed, leading to 26 deaths, highlighting inadequate monitoring of construction defects.

- Mumbai Footbridge Collapse (India, 2019) – A pedestrian bridge collapsed during peak hours, causing multiple casualties due to poor maintenance and undetected cracks.

Each of these incidents underscores the urgent need for a proactive, AI-driven monitoring system that can detect cracks early, prevent failures, and save lives.

Despite the development of AI-based crack detection models, several challenges remain unaddressed:

1. Limited Optimization Studies – Most previous studies focus on older YOLO models (YOLOv4–YOLOv7), without exploring newer architectures (YOLOv8–YOLOv11) that offer improved accuracy and efficiency.
2. Lack of Optimizer Comparisons – Many existing implementations use default training settings, but different optimization techniques (SGD vs. AdamW) can significantly impact performance.
3. Real-World Deployment Issues – Crack detection models often struggle with complex environments, requiring better dataset augmentation and adaptation strategies.
4. Limited UAV Integration – While UAVs (Unmanned Aerial Vehicles) provide efficient access to difficult locations, few studies fully integrate YOLO-based crack detection with UAV monitoring systems.

This research seeks to address these gaps by:

- Evaluating the latest YOLO models (YOLOv8–YOLOv11) for crack detection.
- Implementing SGD and AdamW optimizers to find the best training strategy.
- Integrating the system with UAVs for remote crack detection in bridges, tunnels, and high-rise buildings.

By overcoming these challenges, this study aims to advance automated inspection technologies, reduce human dependency, and enhance infrastructure safety.

1.3 Scope of the Project

The primary objective of this research is to develop and optimize a deep-learning-based crack detection system using YOLO models. This study will:

1. Model Selection & Optimization

- Compare the performance of YOLOv8, YOLOv9, YOLOv10, and YOLOv11.
- Investigate the impact of two optimizers—Stochastic Gradient Descent (SGD) and AdamW—on detection accuracy, inference speed, and mean average precision (mAP).

2. Dataset Collection & Preprocessing

- Utilize high-resolution crack datasets from diverse sources (e.g., buildings, bridges, roads).

- Apply data augmentation (rotation, flipping, scaling, brightness adjustment) to enhance model robustness.

3. Performance Evaluation Metrics

- Measure accuracy based on:
    - mAP (Mean Average Precision) – Evaluates overall detection accuracy.
    - FPS (Frames Per Second) – Determines real-time performance for practical deployment.
    - False Positive & False Negative Rates – Assesses detection reliability.

4. Model Selection for Real-World Deployment & UAV Integration

- Explore the feasibility of deploying YOLO models on UAVs for automated bridge and building inspections.

5. Future Scope

- Enhance detection by integrating multi-modal techniques (e.g., infrared imaging + YOLO) for low-light crack detection.
- Evaluate Transformer-based models (e.g., DETR, SAM) for better feature extraction.
- Develop UAV swarm systems for large-scale automated monitoring.

2. PROJECT DESCRIPTION AND GOALS

2.1 Literature Review

The detection of cracks in infrastructure is a critical aspect of Structural Health Monitoring (SHM). Traditional manual inspection methods are time-consuming, expensive, and prone to human error, which has led to the adoption of automated crack detection systems using deep learning and computer vision techniques. In recent years, You Only Look Once (YOLO) models have emerged as one of the most widely used object detection frameworks for this task due to their high speed and accuracy. This literature review focuses on the evolution of YOLO models in crack detection and identifies the challenges and opportunities in this field.

Deep Learning-Based Crack Detection

In the realm of machine learning, convolutional neural networks (CNNs) have proven to be highly effective for image analysis tasks such as object detection, segmentation, and classification. For crack detection, CNN-based models are trained to identify crack patterns from high-resolution images of concrete surfaces, roads, bridges, and other infrastructure. The primary advantage of using deep learning is its ability to automatically extract features from raw data without requiring manual feature engineering, making the detection process faster, more accurate, and scalable.

Several deep learning models have been applied to crack detection tasks, with notable successes in both small-scale and large-scale infrastructure monitoring. Among these, the YOLO (You Only Look Once) family of models has gained significant popularity due to its real-time detection capability and accuracy in localizing objects in images. YOLO is a single-stage detector, meaning it performs both object classification and localization in one pass, making it faster than many two-stage detectors such as Faster R-CNN.

YOLO in Crack Detection

Initially, YOLO was used for a broad range of object detection tasks, but recent advancements in the YOLO family have allowed for specialized applications in crack detection. YOLO's ability to process images in real-time makes it highly suitable for automated inspection in the field, especially for monitoring large infrastructures such as bridges, tunnels, and high-rise buildings. YOLO has been tested across multiple versions, from YOLOv1 to YOLOv11, with each version introducing improvements in accuracy, processing speed, and handling of complex environments such as varying lighting conditions, occlusions, and different surface textures.

For example, YOLOv4, introduced in 2020, incorporated innovations such as CSPNet, Spatial Pyramid Pooling, and Path Aggregation Networks, which helped the model achieve a 90% accuracy rate in detecting concrete cracks. Similarly, YOLOv5 achieved 99.4% accuracy in detecting cracks in pavement surfaces, showing significant improvements in both speed and feature extraction compared to earlier versions. These improvements have positioned YOLO as a highly effective tool for real-time infrastructure monitoring, providing an alternative to traditional, labor-intensive methods.

Recent research into YOLO versions YOLOv6, YOLOv7, and the latest YOLOv8 and YOLOv9 has shown that these newer models have improved processing speed, better small-object detection capabilities, and enhanced robustness in complex environments. These advancements make YOLO models even more effective for crack detection in historically significant buildings, large infrastructure projects, and urban environments with difficult-to-access areas.

Datasets and Model Training

The accuracy of YOLO-based crack detection systems is heavily reliant on the quality and size of the training datasets. For YOLO to perform effectively, it requires large amounts of high-resolution labeled data, typically involving images with various crack patterns, orientations, and sizes under diverse lighting conditions. In this study, the dataset used is curated from **Roboflow**, consisting of **4029 images** split into training, validation, and test sets. The training set contains **3717 images** (92%), the validation set contains **200 images** (5%), and the test set contains **112 images** (3%).

To further enhance model performance and prevent overfitting, **data augmentation** techniques were applied during the training process. Specifically, for each training example, **three augmented outputs** were generated using various transformations. These augmentations include horizontal flipping, 90° rotations (clockwise, counter-clockwise, and upside down), cropping with a minimum zoom of 0% and maximum zoom of 30%, as well as rotations ranging from **-15° to +15°**. Additional color-related augmentations, such as hue adjustment (within **-10° to +10°**), saturation (within **-25% to +25%**), and brightness (within **-20% to +20%**), were applied. To further enhance the variability in the dataset, **blurring up to 1px** was also utilized.

Moreover, transfer learning techniques were employed, where a pre-trained YOLO model, trained on a general object detection dataset, was fine-tuned for the specific task of crack detection. This approach not only reduces the required training time but also improves model performance, especially with the smaller, more specific dataset.

Challenges in Crack Detection Using YOLO

Despite the promise of YOLO for crack detection, there are several challenges that need to be addressed to improve its practical application:

1. False Positives and False Negatives: While YOLO is effective in detecting cracks, false positives (incorrectly identified cracks) and false negatives (missed cracks) remain an issue, especially when cracks are small, thin, or occluded. These errors can affect the reliability of detection and hinder the system's deployment in real-world scenarios where accuracy is crucial.
2. Environmental Variability: YOLO models may struggle to detect cracks under poor lighting conditions, shadowing, or non-uniform surfaces. In real-world applications, cracks may appear fainter or distorted, and lighting or environmental factors may complicate the detection process, leading to inaccuracies.

3. Computational Resources: Advanced YOLO versions, such as YOLOv8 and YOLOv9, offer impressive detection accuracy and speed. However, these models also require significant computational resources (e.g., high-performance GPUs), making them challenging to deploy in resource-constrained environments such as mobile devices or UAVs.
4. Data Annotation: High-quality labeled datasets are essential for training accurate models. However, the process of manually annotating cracks in large datasets is time-consuming and expensive, requiring expert knowledge. Ensuring that the annotations are consistent and accurate is critical for model performance.

Optimizers and Training Techniques

The training of deep learning models, including YOLO, often involves selecting the best optimizer to ensure efficient convergence and accurate results. Two popular optimizers in deep learning are Stochastic Gradient Descent (SGD) and AdamW. While SGD is known for its stability and simplicity in handling large datasets, AdamW introduces a decoupled weight decay mechanism that enhances model convergence and helps prevent overfitting. Previous studies have shown that AdamW often outperforms SGD in terms of speed and generalization, making it an excellent choice for training YOLO models for crack detection tasks.

Future Directions in YOLO-Based Crack Detection

Several opportunities exist for improving YOLO-based crack detection systems. These include:

- Multi-modal approaches: Combining YOLO with other detection techniques, such as thermal imaging, infrared or radar sensors, can help detect cracks that are invisible to the naked eye or obscured by environmental factors like lighting or surface texture.
- Lightweight YOLO Models: New research on model pruning and quantization could lead to lightweight YOLO versions that can run efficiently on edge devices such as drones, mobile phones, or even small UAVs.
- Integrating UAVs for Large-Scale Inspections: The integration of YOLO-based crack detection with UAVs (Unmanned Aerial Vehicles) has the potential to revolutionize infrastructure monitoring, allowing for quick, remote inspections of hard-to-reach areas without the need for scaffolding or manual inspections.

2.2 Gaps Identified

While existing research on YOLO-based crack detection has shown promising results, several gaps remain in the current state of the field that must be addressed to optimize crack detection systems for real-world applications. These gaps are critical for improving the accuracy, scalability, and efficiency of automated crack detection in infrastructure, particularly in large-scale and complex environments. The following are the key gaps identified in the literature:

1. Inconsistent Performance Across Different Environments

A common challenge in deploying YOLO models for crack detection is the variability in performance across different environments. Cracks in concrete structures may vary in size, orientation, and appearance depending on factors like lighting conditions, surface texture, and occlusions from debris or other objects. Current research tends to focus on datasets captured under controlled conditions, which may not reflect real-world challenges such as poor lighting, extreme weather conditions, or high noise levels. The generalizability of YOLO models across these diverse conditions remains a significant concern.

2. False Positives and False Negatives

Despite the advancements in YOLO models, false positives (incorrectly detecting a crack where there is none) and false negatives (failing to detect an actual crack) continue to be a problem, especially when cracks are small, faint, or partially occluded. This issue is particularly problematic in safety-critical applications like infrastructure monitoring, where missed cracks or incorrectly identified damage could lead to catastrophic failures. Previous research has not sufficiently addressed this issue, and there is a need for more robust algorithms that can better detect minor cracks while reducing the occurrence of false detections.

3. Lack of Adequate and Diverse Datasets

The success of deep learning models such as YOLO depends heavily on the quality and diversity of the datasets used for training. While there are several available datasets for crack detection, they are often limited in terms of image variety, environmental conditions, or crack types. Many existing datasets feature cracks that are uniform in size and shape, collected under ideal lighting conditions. However, real-world infrastructure exhibits cracks of varying sizes, orientations, shapes, and severity, often in complex and hard-to-capture settings. This lack of diversity in training datasets results in models that perform well in specific scenarios but struggle to generalize in others. Moreover, many datasets are not annotated consistently, which further complicates model training and evaluation.

4. Computational and Hardware Constraints

While YOLO is known for its real-time detection capabilities, more advanced versions of YOLO (such as YOLOv8 and YOLOv9) require significant computational resources, including high-performance GPUs. This poses a challenge for the deployment of YOLO-based crack detection systems in resource-constrained environments, such as UAVs (Unmanned Aerial Vehicles), mobile devices, or edge devices. Many studies have not sufficiently explored ways to optimize YOLO models to make them lightweight and capable of running on devices with limited computational power. Additionally, YOLO's high computational demand makes it less practical for large-scale infrastructure inspections where real-time analysis of vast datasets is required.

5. Integration with Other Technologies for Enhanced Detection

While YOLO models have demonstrated strong performance in detecting visible cracks, there is limited research on the integration of YOLO with other sensing technologies to improve

detection in environments where cracks may not be visible or detectable by traditional optical means. For example, thermal imaging, infrared sensing, and laser scanning can detect cracks that are not visible to the naked eye but are critical to structural health. Multi-modal detection systems that combine YOLO with these other technologies are largely unexplored in the context of infrastructure crack detection. Such integrations could greatly enhance the detection capabilities, particularly in challenging environments, like historical structures or complex, multi-layered surfaces.

6. Lack of Real-World Deployment and Long-Term Evaluation

Although many studies have demonstrated the potential of YOLO models for crack detection in controlled environments or datasets, there is a notable lack of research on the real-world deployment of these models for long-term monitoring of infrastructure. The field validation of YOLO models in diverse, real-world conditions is essential to assess the models' reliability, accuracy, and resilience over time. Existing research is often limited to short-term studies or lab-based simulations, which may not reflect the complexities of deploying these models at scale in large, dynamic environments.

7. Insufficient Focus on Cross-Version Comparisons and Optimizer Evaluation

While YOLO versions have seen significant improvements with each iteration, there is a gap in research focused on cross-version comparisons in the context of crack detection. Many studies focus on a single version (e.g., YOLOv5 or YOLOv7), but there is a lack of comprehensive studies comparing the performance of multiple versions (YOLOv8, YOLOv9, YOLOv10, etc.) across a variety of crack types and datasets. Similarly, while optimization algorithms such as Stochastic Gradient Descent (SGD) and AdamW have been explored in deep learning, their comparative performance in YOLO-based crack detection models remains underexplored. There is a need for further research into the choice of optimizers and their impact on convergence speed, accuracy, and robustness in crack detection tasks.

8. Limited Exploration of Transfer Learning and Fine-Tuning

Transfer learning has shown significant potential for improving model performance, especially when the available training data is limited. However, there is limited research on transfer learning in the context of YOLO-based crack detection. Pre-trained models (trained on general object detection tasks) could be fine-tuned for crack detection tasks, allowing models to learn faster and generalize better from smaller datasets. Although some research has explored transfer learning for crack detection, there is a lack of studies that rigorously evaluate the effectiveness of various pre-training strategies and fine-tuning techniques across multiple YOLO versions.

2.3 Objectives

The primary objective of this research is to develop an optimal, automated crack detection system using the You Only Look Once (YOLO) family of models—YOLOv8, YOLOv9,

YOLOv10, and YOLOv11—to enhance Structural Health Monitoring (SHM) for critical infrastructure. The main goals of this project are outlined below:

1. Develop an Advanced Automated Crack Detection System

The first objective is to design and implement an automated crack detection system using the YOLO family of models, which are known for their real-time detection and high accuracy in object recognition. The system will be trained on diverse datasets to detect a wide variety of crack patterns in infrastructure (such as bridges, buildings, and historical structures) under various real-world conditions. This will ensure that the system is effective in different environments and adaptable to challenging scenarios.

2. Evaluate and Compare Different YOLO Versions for Optimal Performance

Another objective is to evaluate the performance of multiple YOLO versions—YOLOv8, YOLOv9, YOLOv10, and YOLOv11—in detecting cracks on infrastructure surfaces. This includes comparing their detection accuracy, inference speed, and mean average precision (mAP), across different crack types, sizes, and orientations. The goal is to determine the most suitable YOLO version that can provide high detection accuracy while maintaining real-time processing speeds for large-scale infrastructure monitoring.

3. Optimize YOLO Models with Advanced Optimization Algorithms

To enhance the model's accuracy and efficiency, this research aims to implement and compare two popular optimization algorithms—Stochastic Gradient Descent (SGD) and AdamW. These optimizers will be evaluated to understand how they affect model convergence, accuracy, and generalization in the context of crack detection. The goal is to determine which optimization technique best suits YOLO models for infrastructure applications, considering factors such as training time, stability, and performance.

4. Enhance the Model’s Adaptability Using Transfer Learning

This objective focuses on leveraging transfer learning to improve the performance of YOLO models, especially in scenarios where training data is limited. By using pre-trained models from general object detection tasks, the goal is to enhance the model’s ability to detect cracks with fewer training images, ensuring faster deployment and better generalization to new, unseen datasets. This will make the model more efficient in real-world applications, where labeled data may not be abundant.

5. Implement Data Augmentation Techniques to Improve Generalization

The research aims to improve the model's generalization ability by applying data augmentation techniques such as rotation, scaling, flipping, and cropping on the input datasets. These techniques will simulate various environmental conditions (e.g., lighting variations, occlusions, and background complexities) that may be encountered during real-world crack detection. The objective is to make the model more robust to diverse and challenging conditions, ultimately increasing its accuracy and reliability for long-term infrastructure monitoring.

6. Integrate the Optimized YOLO Model with UAVs for Remote Detection

The project will explore the integration of the optimized YOLO-based crack detection system with Unmanned Aerial Vehicles (UAVs) for remote and inaccessible area inspections. This objective aims to develop a scalable, mobile solution for automated crack detection, enabling real-time data collection from hard-to-reach areas. The goal is to design an easy-to-deploy system that can quickly assess infrastructure in danger zones or locations with limited access, thus enhancing both efficiency and safety in structural health monitoring.

7. Conduct a Comparative Analysis to Determine the Optimal System Configuration

Finally, the project seeks to perform a comprehensive comparative analysis of different system configurations to determine the optimal setup for real-time crack detection. This will involve testing various hyperparameters, model architectures, and deployment methods, focusing on balancing the trade-off between precision and efficiency. The ultimate goal is to establish an end-to-end crack detection system that is suitable for large-scale, real-time applications in infrastructure monitoring.

2.4 Problem Statement

Infrastructure, particularly critical structures like bridges, buildings, and roads, plays a pivotal role in ensuring the safety and functionality of society. Over time, these structures undergo wear and tear due to various environmental, mechanical, and physical stresses. One of the most common forms of damage is the formation of surface cracks in concrete, which can compromise the structural integrity of buildings and bridges. If these cracks go unnoticed or are not addressed in a timely manner, they can escalate into severe failures, resulting in loss of life, property damage, and high repair costs.

Traditional methods of crack detection—such as visual inspections, manual assessments, and basic non-destructive testing techniques—are often time-consuming, labor-intensive, costly, and prone to human error. Inspectors might miss subtle or hidden cracks, particularly those in hard-to-reach locations or those that are not immediately visible. Moreover, human error can introduce inconsistencies and lead to inaccurate assessments. This significantly undermines the reliability of infrastructure monitoring systems, especially for large-scale and high-risk structures.

As seen in several high-profile incidents, including the 2018 Morandi Bridge collapse in Italy and the 2021 Champlain Towers collapse in the United States, unnoticed cracks and neglected infrastructure monitoring can lead to disastrous consequences. These events highlight the urgency of transitioning from traditional, manual inspection methods to automated, accurate, and efficient systems for detecting cracks in infrastructure.

Challenges in Current Crack Detection Methods

While advancements in technology have led to the development of various automated crack detection systems, many of these solutions still face significant challenges. One of the most prominent challenges is the accuracy of detection, particularly in real-world environments where conditions can vary drastically. For instance, cracks in concrete structures may present

in various forms—small or large, shallow or deep, horizontal or vertical—and under different environmental conditions, such as varying light, weather, and background complexities. Current deep learning models, especially those based on the You Only Look Once (YOLO) framework, have shown promising results, but they still struggle with false positives, false negatives, and the ability to generalize across a wide range of conditions.

Another challenge is the computational cost of training and deploying these models. Advanced YOLO versions like YOLOv8 and YOLOv9 often require high-performance GPUs for both training and real-time inference, which limits their practical application in environments with limited resources, such as Unmanned Aerial Vehicles (UAVs) or mobile devices. Furthermore, while YOLO models can perform real-time crack detection, they often struggle with the detection of small cracks, cracks with complex orientations, or cracks that are obscured by debris, which are common in real-world infrastructure.

In addition, the lack of diverse, high-quality datasets—especially those that simulate the variety of crack patterns and environmental conditions encountered in practical applications—poses a significant limitation for YOLO-based models. Most existing datasets are small, narrowly focused on specific crack types, and lack sufficient data augmentation to account for real-world complexities. This limits the effectiveness and generalization ability of trained models.

Finally, the integration of YOLO-based models with other technologies, such as thermal imaging, infrared sensing, or laser scanning, remains an underexplored area. These technologies can help detect cracks that are invisible to the naked eye or located beneath surface layers, but integrating them with real-time deep learning models like YOLO could lead to more robust, multi-modal detection systems. However, such integrations are rare, and there is insufficient research into how they can be effectively implemented.

The Problem to Address

This research seeks to address these challenges by developing an automated crack detection system that combines high-performance YOLO models with advanced optimization techniques and real-world data augmentation strategies. The core problem is to optimize YOLO's performance for real-time crack detection in dynamic, complex environments, while addressing issues like accuracy, computational efficiency, and model generalization.

The key problems this research aims to solve are:

1. Inconsistent Detection Across Different Environments: YOLO models may not perform consistently in real-world conditions, such as poor lighting, occlusions, or varying crack types and orientations.
2. False Positives and False Negatives: There is a need to reduce the rate of false detections, where the model either incorrectly identifies cracks or misses actual cracks, particularly in small or partially occluded cracks.
3. Scalability and Computational Constraints: Optimizing YOLO models to perform well with limited computational resources, such as in UAVs or on mobile devices, is crucial for large-scale infrastructure monitoring.

4. Lack of Real-World Deployment and Long-Term Validation: The need for models that are not only effective in controlled environments but also capable of real-world deployment for continuous, long-term infrastructure monitoring.
5. Lack of Multi-Modal Detection Systems: Integration of YOLO with other advanced sensing technologies, such as infrared, thermal, or laser imaging, to detect cracks that are not visible to the naked eye, thus providing a more comprehensive solution.

By addressing these challenges, this research aims to make significant contributions to proactive infrastructure monitoring, helping to ensure the safety and longevity of critical infrastructure. This system will reduce the need for costly manual inspections and increase the overall reliability and efficiency of crack detection in buildings, bridges, and other vital structures.

2.5 Project Plan

The Project Plan outlines the structured process and steps to achieve the objectives of the automated crack detection system using YOLO-based models for Structural Health Monitoring (SHM). The plan ensures a systematic approach from research, data collection, and model development to testing, deployment, and documentation. Each stage is designed to guarantee the successful development and implementation of the system.

1. Research and Literature Review

Objective: To understand the state-of-the-art crack detection technologies, identify gaps, and refine the project scope.

Activities:

- Comprehensive Literature Review: Analyze current research papers, case studies, and materials related to automated crack detection, YOLO models, and their applications in infrastructure monitoring.
- Gap Identification: Evaluate existing methods, focusing on limitations such as accuracy, model optimization, and integration issues.
- Scope Refinement: Finalize the project's research focus, ensuring it aligns with both academic trends and practical needs in the infrastructure sector.

Deliverables:

- Detailed literature review report.
- A list of identified gaps and areas for improvement.

2. Data Collection and Preprocessing

Objective: To collect, preprocess, and augment a dataset of crack images that will be used for model training and evaluation.

Activities:

- Dataset Collection: Gather diverse crack images from public datasets or create a custom dataset by collecting images from various infrastructure, including concrete roads, bridges, and historical structures.
- Data Augmentation: Apply techniques like rotation, flipping, and scaling to increase dataset diversity and improve model robustness.
- Data Labeling: Annotate images with bounding boxes and labels, ensuring the cracks are clearly marked.
- Data Splitting: Divide the data into training, validation, and test sets for model development and evaluation.

Deliverables:

- Annotated and augmented dataset ready for model training.
- Data preprocessing pipeline documentation.

3. Model Selection and Optimization

Objective: To evaluate and optimize different versions of YOLO models for crack detection, ensuring high accuracy and performance.

Activities:

- Model Selection: Choose among YOLO versions (e.g., YOLOv8, YOLOv9, YOLOv10, YOLOv11) based on their capabilities in small object detection, handling occlusions, and real-time processing.
- Implementation of Optimizers: Utilize Stochastic Gradient Descent (SGD) and AdamW optimizers to determine which one yields better convergence and accuracy for the models.
- Transfer Learning: Fine-tune pre-trained YOLO models on the crack dataset to accelerate the training process and improve generalization.
- Hyperparameter Tuning: Adjust hyperparameters, such as learning rates, batch sizes, and epoch counts, to achieve optimal model performance.
- Evaluation Metrics: Assess models based on Mean Average Precision (mAP), Precision, Recall, F1-Score, and Inference Speed.

Deliverables:

- Trained YOLO models with optimized parameters.
- Performance evaluation report for different YOLO models and optimizers.

4. Model Evaluation and Validation

Objective: To rigorously test and validate the models under real-world conditions to ensure reliability.

Activities:

- Testing on Unseen Data: Evaluate the models on a separate test dataset to assess their generalization ability.
- Real-World Validation: Simulate deployment conditions by testing models in various settings with different environmental factors like lighting conditions, occlusions, and backgrounds.
- Error Analysis: Identify and analyze common errors (e.g., false positives or false negatives) and refine the model to minimize these issues.

Deliverables:

- Final evaluation report detailing model performance.
- Error analysis and model improvements based on the validation results.

5. UAV Integration and Deployment

Objective: To integrate the YOLO-based crack detection system with Unmanned Aerial Vehicles (UAVs), enabling remote inspections in hard-to-reach areas.

Activities:

- UAV Platform Selection: Choose an appropriate UAV with sufficient computational resources and camera specifications to support deep learning-based crack detection.
- Model Deployment: Deploy the trained YOLO model on the UAV, allowing it to perform crack detection in real-time while flying over infrastructure.
- Flight Testing: Conduct field tests to evaluate the UAV system's real-time performance and its ability to detect cracks in diverse conditions.
- System Optimization: Focus on improving inference speed to ensure that the UAV can detect cracks in real-time without lag, enabling efficient inspections.

Deliverables:

- UAV-integrated crack detection system.
- Data and results from flight testing, including performance metrics.

6. Final Evaluation and Documentation

Objective: To compile the results, analyze the outcomes, and document the research for dissemination.

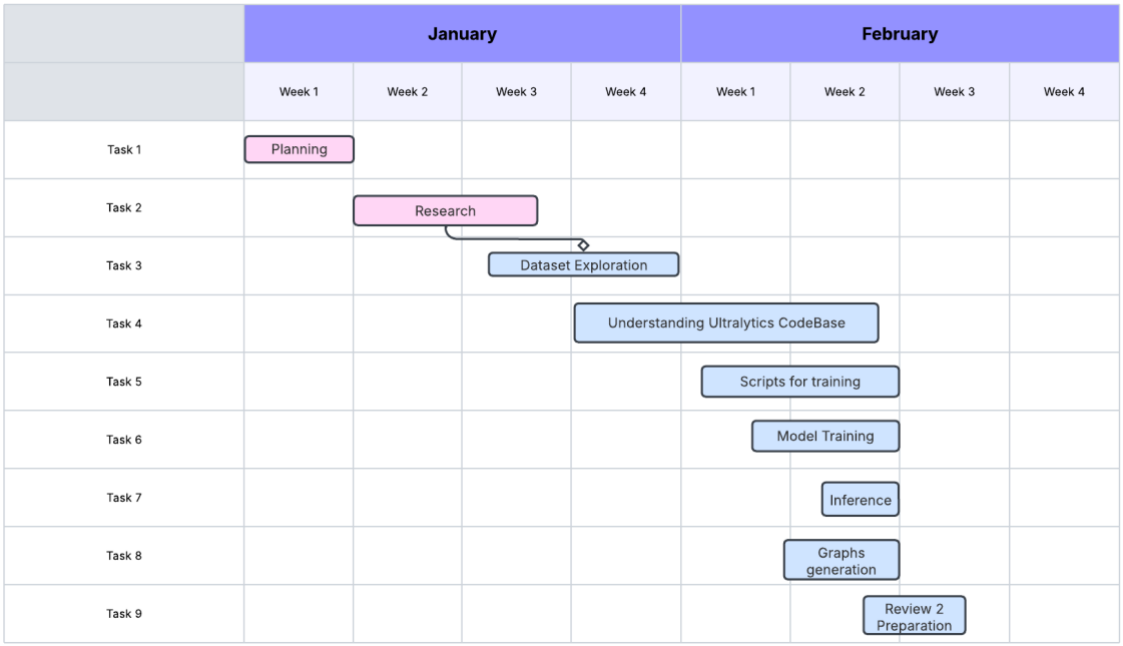
Activities:

- Final Testing: Conduct a final round of tests to confirm that the models and UAV system meet the project objectives.

- Report Writing: Document the entire process, including methodologies, results, and conclusions. Summarize findings in a comprehensive research paper.
- Presentation Preparation: Prepare presentation materials for conferences, stakeholders, or potential funders to showcase the project.
- Publishing: Submit the final paper to academic journals or conferences for peer review and publication.

Deliverables:

- Completed research paper detailing findings and conclusions.
- Presentation slides for research dissemination.



3. Requirement Analysis

3.1 Software and Hardware Requirements

The development and deployment of a YOLO-based crack detection system require specific software and hardware configurations to ensure effective training, model evaluation, and real-time performance. The following outlines the **hardware and software requirements** that are integral to the success of this study, along with the justification for their selection.

Hardware: GPU T4-16GB

For the training of deep learning models like YOLO, computational power plays a critical role in both the **speed of training** and the **accuracy of the results**. The chosen **GPU T4-16GB** (a high-performance graphics processing unit) is essential for accelerating the computation-heavy tasks associated with deep learning. With **16GB of VRAM**, this GPU offers substantial memory capacity, allowing the model to process large, high-resolution images typical of crack detection tasks. GPUs are designed to handle the **parallel processing** demands of neural

networks, significantly reducing the time required for **forward and backward propagation** during training compared to CPU-based computations.

The T4 GPU also supports TensorFlow and PyTorch, two of the most widely used frameworks for training deep learning models, making it highly compatible with the **Ultralytics YOLO codebase** used in this study. Additionally, its efficient architecture provides a cost-effective balance between performance and resource utilization, making it ideal for **real-time deployment** and **fine-tuning large models** such as YOLOv8 and YOLOv10, which require heavy computation for accurate crack detection.

Software: Google Colab

Google Colab is a cloud-based environment that allows developers to run Python code on powerful GPUs without the need for local hardware infrastructure. It offers easy access to the **GPU T4-16GB** and supports libraries like TensorFlow, PyTorch, and other machine learning frameworks needed to train and test YOLO models. Colab's collaborative nature allows for the seamless sharing of code and results, enhancing research collaboration.

By using Google Colab, the study benefits from the **free access to GPU acceleration**, making it an ideal platform for handling the intense computational requirements of YOLO model training. Moreover, Colab provides **easy integration with Google Drive**, which facilitates efficient storage and retrieval of large datasets such as the one used in this study. It ensures that the entire development process, from preprocessing to model evaluation, is streamlined in a cloud environment that can be accessed from anywhere, without the need for dedicated local resources.

Software: Ultralytics CodeBase

The **Ultralytics YOLO codebase** is one of the most popular and robust implementations of the YOLO algorithm. It is highly optimized for **real-time object detection** and supports a variety of YOLO versions, including YOLOv5, YOLOv7, and YOLOv8, which are essential for this study's goal of evaluating multiple YOLO versions for crack detection. The codebase is specifically designed to be **user-friendly**, providing built-in features for dataset management, model training, testing, and evaluation.

Ultralytics also ensures compatibility with modern deep learning libraries, facilitating **easy integration with hardware accelerators like GPUs**. By leveraging this codebase, the research can benefit from the **highly optimized training pipeline**, which ensures faster convergence and higher accuracy for crack detection tasks. It also supports **hyperparameter tuning**, allowing the model to be adjusted for optimal crack detection performance under different conditions.

Software: Roboflow Library

The **Roboflow library** is used for dataset preparation, augmentation, and model deployment. It allows users to easily upload, annotate, and enhance datasets, which is particularly useful for crack detection, where the dataset needs to contain a wide range of crack patterns, orientations, and environments. Roboflow's integration with deep learning frameworks ensures seamless **data augmentation**, which helps prevent overfitting and increases the diversity of the dataset by applying transformations such as rotations, flips, color adjustments, and zooms.

Roboflow also enables **collaborative dataset management**, making it easier to share datasets and pre-trained models across different teams or research settings. The library provides **easy model export options**, which can be directly integrated into training pipelines on platforms like Google Colab. For crack detection, it ensures that the dataset is not only of high quality but also optimized for training, which is crucial for achieving **high accuracy** with YOLO models.

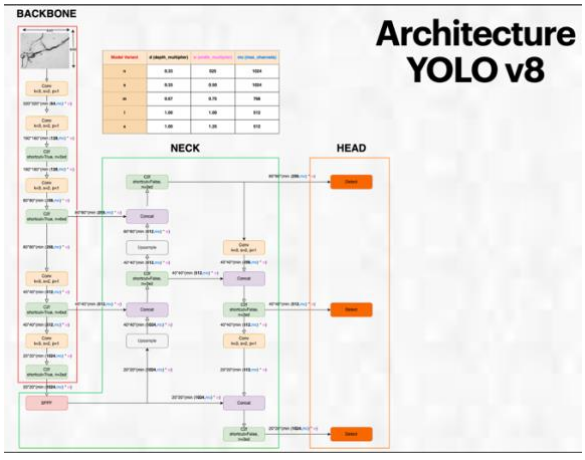
Software: Kaggle

Finally, **Kaggle** serves as an additional platform for managing datasets and leveraging pre-built kernels for model development. It offers access to numerous datasets relevant to crack detection and other civil engineering applications, which can be used for initial exploration or benchmarking. Kaggle's integration with Google Cloud makes it easier to leverage cloud-based GPU instances, similar to **Google Colab**, and provides a **centralized environment** for storing and sharing datasets, model code, and results.

3.2 Models Required

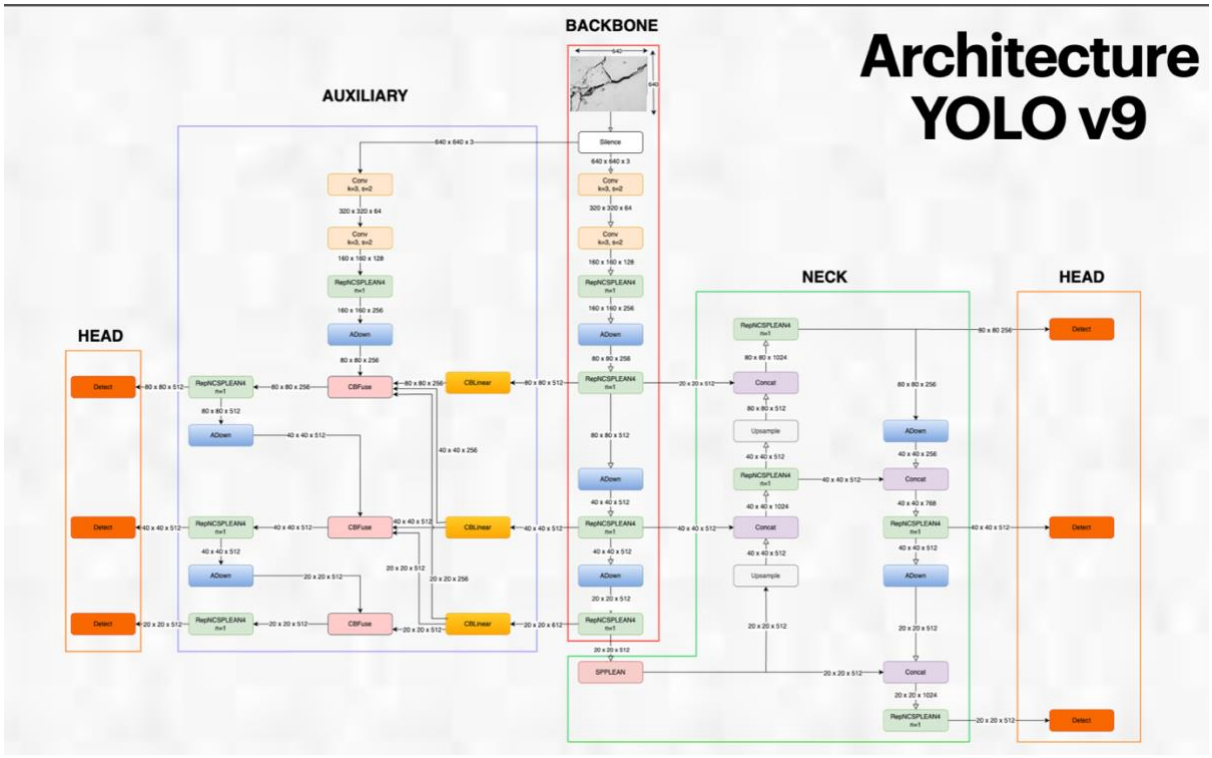
YOLOv8

YOLOv8 (You Only Look Once version 8) represents the latest advancement in the YOLO family of object detection models. It brings significant improvements in both accuracy and speed, making it highly suitable for real-time applications such as crack detection in civil infrastructure. One of the key features of YOLOv8 is its lightweight architecture, which enhances its performance on edge devices without compromising detection accuracy. The model introduces improved backbone networks, which allow for better feature extraction, leading to finer details in images, such as small cracks or defects. YOLOv8 is designed to handle complex real-world conditions, offering better robustness against background noise, occlusions, and variations in lighting. With its single-stage detection mechanism, YOLOv8 processes images quickly, making it ideal for real-time monitoring systems, such as those used in infrastructure health assessments or UAV-based inspection systems. Additionally, the model benefits from extensive hyperparameter optimization tools that ensure improved detection performance even when trained on diverse datasets with varying crack patterns and surface materials.

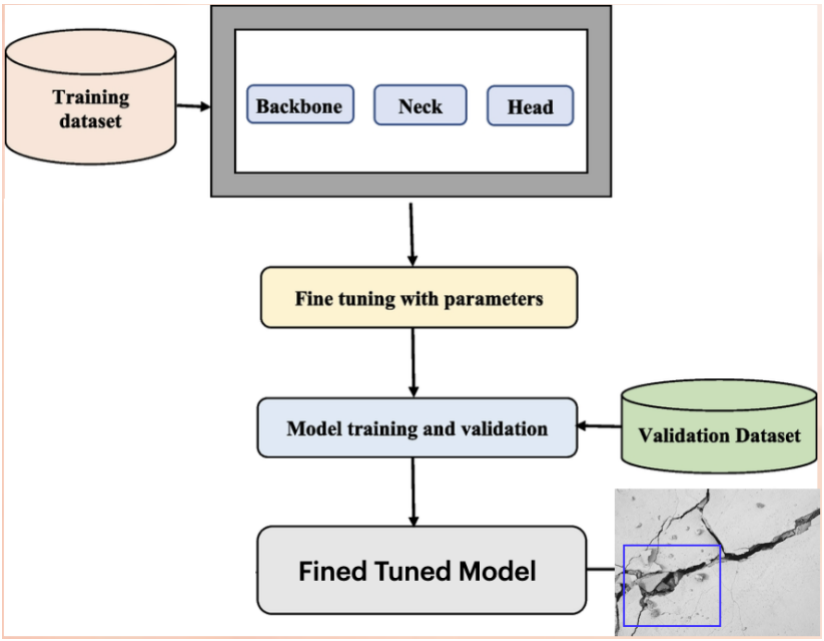


YOLOv9

YOLOv9, although still in development or in preliminary releases, builds upon the foundation laid by YOLOv8, further optimizing detection accuracy and efficiency. This version aims to address some of the remaining challenges in complex environments, such as dealing with small object detection and multi-scale features. YOLOv9 enhances the model's ability to detect fine cracks or subtle defects in large infrastructure projects by incorporating advanced techniques like attention mechanisms and improved anchor boxes. The architecture has been streamlined for faster inference times, making it more suitable for edge computing applications or embedded systems where computational resources are limited. YOLOv9 also incorporates advancements in transfer learning, allowing for quicker adaptation to specialized tasks such as crack detection with fewer labeled samples. This makes it an appealing choice for situations where dataset limitations or real-time processing constraints are significant factors. With improved detection capabilities in challenging scenarios, YOLOv9 could provide significant benefits in fields requiring accurate, fast, and scalable object detection, such as construction site monitoring or automated structural health inspections.



4. System Design



This system design represents the process of fine-tuning a YOLO (You Only Look Once) model for object detection, specifically for tasks like crack detection (as inferred from the example image at the bottom right). Let's break down each component and its role in fine-tuning the YOLO model.

The diagram depicts a structured workflow for training and fine-tuning a YOLO model. It starts with the training dataset, processes it through the model's backbone, neck, and head, fine-tunes the parameters, trains and validates the model, and finally produces a fine-tuned model that can detect objects (e.g., cracks in images).

A. Training Dataset

- The process begins with a dataset containing labeled images, which are used for training the YOLO model.
- This dataset includes images annotated with bounding boxes around the target objects (e.g., cracks in a structural inspection scenario).
- The dataset is formatted as per YOLO requirements (e.g., using .txt label files with class and bounding box coordinates).

B. YOLO Model Structure

The YOLO model is divided into three key components:

1. **Backbone**
    - Extracts feature representations from the input image.

- Commonly used backbones in YOLO versions include CSPDarknet, MobileNet, or EfficientNet.
- It processes raw pixel data into meaningful feature maps.

2. **Neck**
   - Bridges the backbone and the head.
   - Enhances multi-scale feature representation, allowing the model to detect objects of varying sizes.
   - Common architectures include PANet (Path Aggregation Network) and FPN (Feature Pyramid Network).
3. **Head**
   - Outputs the final predictions (bounding box coordinates, class labels, and confidence scores).
   - Uses anchor-based or anchor-free detection strategies.
   - Applies non-max suppression (NMS) to refine predictions.

C. Fine-Tuning with Parameters

- Fine-tuning involves adjusting the model's hyperparameters to improve detection accuracy.
- Parameters include:
  - Learning rate (affects how quickly the model updates weights).
  - Batch size (number of images processed at once).
  - Number of epochs (how many times the dataset is processed).
  - Data augmentation techniques (flipping, rotation, color adjustments).
  - Anchor box tuning (modifying default bounding box sizes).

D. Model Training and Validation

- The model is trained using the processed training dataset.
- It learns object features by minimizing loss functions such as:
  - **Bounding box loss** (measures accuracy of predicted vs. actual boxes).
  - **Classification loss** (ensures correct object identification).
  - **Confidence loss** (assesses certainty of object presence).
- Validation dataset is used to check the model's generalization ability, ensuring it doesn't overfit the training data.

E. Fine-Tuned Model

- The final trained model can detect objects in new images.
- It generalizes well to unseen data, providing accurate predictions

REFERENCES

1. Mohan, A.; Poobal, S. Crack detection using image processing: A critical review and analysis. Alex. Eng. J. 2018, 57, 787–798. [CrossRef]
2. Zaloshnja, E.; Miller, T.R. Cost of crashes related to road conditions, United States, 2006. In Proceedings of the Annals of Advancesin Automotive Medicine/Annual Scientific Conference, Baltimore, Maryland, 5–7 October 2009; Association for the Advancement of Automotive Medicine: Chicago, IL, USA, 2009.
3. Zhu, Z.; German, S.; Brilakis, I. Visual retrieval of concrete crack properties for automated post-earthquake structural safety evaluation. Autom. Constr. 2011, 20, 874–883. [CrossRef]
4. Gkantou, M.; Muradov, M.; Kamaris, G.S.; Hashim, K.; Atherton, W.; Kot, P. Novel Electromagnetic Sensors Embedded in Reinforced Concrete Beams for Crack Detection. Sensors 2019, 19, 5175. [CrossRef]
5. Koshti, A.M. X-ray ray tracing simulation and flaw parameters for crack detection. In Health Monitoring of Structural and Biological Systems XII; SPIE: Denver, CO, USA, 2018.
6. Hosseini, Z.; Momayez, M.; Hassani, F.; Lévesque, D. Detection of inclined cracks inside concrete structures by ultrasonic SAFT. In AIP Conference Proceedings; American Institute of Physics: College Park, MD, USA, 2008.
7. Rodríguez-Martin, M.; Lagüela, S.; González-Aguilera, D.; Arias, P. Cooling analysis of welded materials for crack detection using infrared thermography. Infra-Red Phys. Technol. 2014, 67, 547–554. [CrossRef]
8. Mahler, D.S.; Kharoufa, Z.B.; Wong, E.K.; Shaw, L.G. Pavement distress analysis using image processing techniques. Comput. Aided Civ. Infrastruct. Eng. 1991, 6, 1–14. [CrossRef]
9. Oliveira, H.; Correia, P.L. Automatic road crack segmentation using entropy and image dynamic thresholding. In Proceedings of the 17th European Signal Processing Conference, Scotland, UK, 24–28 August 2009.
10. Huang, Y.; Xu, B. Automatic inspection of pavement cracking distress. J. Electron. Imaging 2006, 15, 013017. [CrossRef]
11. Otsu, N. A Threshold Selection Method from Gray-Level Histograms. IEEE Trans. Syst. Man Cybern. 1979, 9, 62–66. [CrossRef]
12. Liu, Y., W. Gao, T. Zhao, Z. Wang, and Z. Wang. A Rapid Bridge Crack Detection Method Based on Deep Learning. Applied Sciences, Vol. 13, No. 17, 2023, p. 9878. https://doi.org/10.3390/app13179878.
13. Su, H., D. B. Kamanda, T. Han, C. Guo, R. Li, Z. Liu, F. Su, and L. Shang. Enhanced YOLO v3 for Precise Detection of Apparent Damage on Bridges amidst Complex Backgrounds. Scientific Reports, Vol. 14, No. 1, 2024, p. 8627. https://doi.org/10.1038/s41598-024-58707-2.
14. Yang, Y., L. Li, G. Yao, H. Du, Y. Chen, and L. Wu. An Modified Intelligent Real-Time Crack Detection Method for Bridge Based on Improved Target Detection Algorithm and Transfer Learning. Frontiers in Materials, Vol. 11, 2024. https://doi.org/10.3389/fmats.2024.1351938.

15. Xiong, C., T. Zayed, and E. M. Abdelkader. A Novel YOLOv8-GAM-Wise-IoU Model for Automated Detection of Bridge Surface Cracks. Construction and Building Materials, Vol. 414, 2024, p. 135025. https://doi.org/10.1016/j.conbuildmat.2024.135025.
16. Li, T., G. Liu, and S. Tan. Superficial Defect Detection for Concrete Bridges Using YOLOv8 with Attention Mechanism and Deformation Convolution. Applied Sciences, Vol. 14, No. 13, 2024, p.5497. https://doi.org/10.3390/app14135497.
17. Matarneh, S., F. Elghaish, F. Pour Rahimian, E. Abdellatef, and S. Abrishami. Evaluation and Optimisation of Pre-Trained CNN Models for Asphalt Pavement Crack Detection and Classification. Automation in Construction, Vol. 160, 2024, p. 105297. https://doi.org/10.1016/j.autcon.2024.105297.
18. Zhang, Q., S. Chen, Y. Wu, Z. Ji, F. Yan, S. Huang, and Y. Liu. Improved U-Net Network Asphalt Pavement Crack Detection Method. PLOS ONE, Vol. 19, No. 5, 2024, p. e0300679. https://doi.org/10.1371/journal.pone.0300679.
19. Qin, Z., Z. Li, Z. Zhang, Y. Bao, G. Yu, Y. Peng, and J. Sun. ThunderNet: Towards Real-Time Generic Object Detection. 2019.
20. Su, P., H. Han, M. Liu, T. Yang, and S. Liu. MOD-YOLO: Rethinking the YOLO Architecture at the Level of Feature Information and Applying It to Crack Detection. Expert Systems with Applications, Vol. 237, 2024, p. 121346. https://doi.org/10.1016/j.eswa.2023.121346.
21. Xiong, C., T. Zayed, X. Jiang, G. Alfalah, and E. M. Abelkader. A Novel Model for Instance Segmentation and Quantification of Bridge Surface Cracks—The YOLOv8-AFPN-MPD-IoU. Sensors, Vol. 24, No. 13, 2024, p. 4288. https://doi.org/10.3390/s24134288.
22. Tang, G., C. Yin, X. Zhang, X. Liu, and S. Li. Crack-Detection Method for Asphalt Pavement Based on the Improved YOLOv5. Journal of Performance of Constructed Facilities, Vol. 38, No. 2, 2024. https://doi.org/10.1061/JPCFEV.CFENG-4615.
23. Liu, Q., and Z. Liu. Asphalt Pavement Crack Detection Based on Improved YOLOv5s Algorithm. 2024.
24. Tran, T. S., S. D. Nguyen, H. J. Lee, and V. P. Tran. Advanced Crack Detection and Segmentation on Bridge Decks Using Deep Learning. Construction and Building Materials, Vol. 400, 2023, p. 132839. https://doi.org/10.1016/j.conbuildmat.2023.132839.
25. Dwyer, B. , N. J. , H. T. , et. al. Roboflow (Version 1.0) [Software]. Computer Vision.
26. Glenn Jocher, A. C. J. Q. Ultralytics YOLOv8. https://github.com/ultralytics/ultralytics. Accessed Nov. 18, 2024.
27. Li, C., L. Li, H. Jiang, K. Weng, Y. Geng, L. Li, Z. Ke, Q. Li, M. Cheng, W. Nie, Y. Li, B. Zhang, Y. Liang, L. Zhou, X. Xu, X. Chu, X. Wei, and X. Wei. YOLOv6: A Single-Stage Object Detection Framework for Industrial Applications. 2022.
28. Bochkovskiy, A., C.-Y. Wang, and H.-Y. M. Liao. YOLOv4: Optimal Speed and Accuracy of Object Detection. 2020.
29. Wang, C.-Y., A. Bochkovskiy, and H.-Y. M. Liao. YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors. 2022.
30. Redmon, J., and A. Farhadi. YOLOv3: An Incremental Improvement. 2018.
31. Redmon, J., and A. Farhadi. YOLO9000: Better, Faster, Stronger. 2017.

32. Ramos, L., E. Casas, E. Bendek, C. Romero, and F. Rivas-Echeverría. Hyperparameter Optimization of YOLOv8 for Smoke and Wildfire Detection: Implications for Agricultural and Environmental Safety. Artificial Intelligence in Agriculture, Vol. 12, 2024, pp. 109–126. https://doi.org/10.1016/j.aiia.2024.05.003.
33. Chen, X., C. Yuan, and Y. Liu. Research On Bird Nest Detection Method of Transmission Line Based on Yolov8. International Journal of Computer Science and Information Technology, Vol. 3, No. 2, 2024, pp. 336–344. https://doi.org/10.62051/ijcsit.v3n2.36.
34. Robbins, H., and S. Monro. A Stochastic Approximation Method. 1951.
35. Kingma, D. P., and J. Lei Ba. ADAM: A Method for Stochastic Optimization. Presented at 3rd International Conference for Learning Representations, San Diego, 2015.
36. Loshchilov, I., and F. Hutter. Decoupled Weight Decay Regularization. Presented at the 9th International Conference for Learning Representations , New Orleans, 2019.
37. Kurbiel, T., and S. Khaleghian. Training of Deep Neural Networks Based on Distance Measures Using RMSProp. 2017.
38. Liu, L., G. Tech, P. He, W. Chen, X. Liu, J. Gao, and J. Han. On the Variance of the Adaptive Learning Rate and Beyond. Presented at the 10th International Conference for Learning Representations, 2020.
39. Timothy Dozat. Incorporating Nesterov Momentum into Adam. Presented at 4th International Conference for Learning Representations, San Juan, 2016.
40. Czitrom, V. One-Factor-at-a-Time versus Designed Experiments. The American Statistician, Vol. 53, No. 2, 1999, pp. 126–131. https://doi.org/10.1080/00031305.1999.10474445.
41. Taffese, W. Z., and L. Espinosa-Leal. Prediction of Chloride Resistance Level of Concrete Using Machine Learning for Durability and Service Life Assessment of Building Structures. Journal of Building Engineering, Vol. 60, 2022, p. 105146. https://doi.org/10.1016/j.jobe.2022.105146.
42. Karras, T., S. Laine, and T. Aila. A Style-Based Generator Architecture for Generative Adversarial Networks. 2018. 43. Dosovitskiy, A., J. T. Springenberg, M. Tatarchenko, and T. Brox. Learning to Generate Chairs, Tables and Cars with Convolutional Networks. 2014. 44. Kingma, D. P., and M. Welling. Auto-Encoding Variational Bayes. 2013.
43. Li, R., J. Yu, F. Li, R. Yang, Y. Wang, and Z. Peng. Automatic Bridge Crack Detection Using Unmanned Aerial Vehicle and Faster R-CNN. Construction and Building Materials, Vol. 362, 2023, p. 129659. https://doi.org/10.1016/j.conbuildmat.2022.129659.
44. Lu, G., X. He, Q. Wang, F. Shao, J. Wang, and Q. Jiang. Bridge Crack Detection Based on Improved Single Shot Multi-Box Detector. PLOS ONE, Vol. 17, No. 10, 2022, p. e0275538. https://doi.org/10.1371/journal.pone.0275538.
45. He, M., and T. L. Lau. CrackHAM: A Novel Automatic Crack Detection Network Based on U-Net for Asphalt Pavement. IEEE Access, Vol. 12, 2024, pp. 12655–12666. https://doi.org/10.1109/ACCESS.2024.3353729.
46. Zoubir, H., M. Rguig, M. El Aroussi, R. Saadane, and A. Chehri. Pixel-Level Concrete Bridge Crack Detection Using Convolutional Neural Networks, Gabor Filters, and Attention Mechanisms. Engineering Structures, Vol. 314, 2024, p. 118343. https://doi.org/10.1016/j.engstruct.2024.118343.

47. Redmon, J., S. Divvala, R. Girshick, and A. Farhadi. You Only Look Once: Unified, Real-Time Object Detection. 2016.
48. Gong, H., J. Tešić, J. Tao, X. Luo, and F. Wang. Automated Pavement Crack Detection with Deep Learning Methods: What Are the Main Factors and How to Improve the Performance? Transportation Research Record: Journal of the Transportation Research Board, Vol. 2677, No. 10, 2023, pp. 311–323. https://doi.org/10.1177/03611981231161358.
49. Zhang, J., S. Qian, and C. Tan. Automated Bridge Crack Detection Method Based on Lightweight Vision Models. Complex & Intelligent Systems, Vol. 9, No. 2, 2023, pp. 1639–1652. https://doi.org/10.1007/s40747-022-00876-6.
50. Zhang, J.; Jing, J.; Lu, P.; Song, S. Improved MobileNetV2-SSDLite for automatic fabric defect detection system based on cloud-edge computing. Measurement 2022, 201, 111665. [CrossRef]
51. Zhang, J.; Jing, J.; Lu, P.; Song, S. Concrete crack segmentation based on UAV-enabled edge computing. Neurocomputing 2022, 485, 233–241.
52. Tang, W.; Yang, Q.; Hu, X.; Yan, W. Deep learning-based linear defects detection system for large-scale photovoltaic plants based on an edge-cloud computing infrastructure. Sol. Energy 2022, 231, 527–535. [CrossRef]
53. Wu, Y.; Guo, H.; Chakraborty, C.; Khosravi, M.; Berretti, S.; Wan, S. Edge Computing Driven Low-Light Image Dynamic Enhancement for Object Detection. IEEE Trans. Netw. Sci. Eng. 2022. [CrossRef]
54. Zhu, T.; Kuang, L.; Daniels, J.; Herrero, P.; Li, K.; Georgiou, P. IoMT-Enabled Real-time Blood Glucose Prediction with Deep Learning and Edge Computing. IEEE Internet Things J. 2022, 10, 3706–3719. [CrossRef]
55. Zhang, J.; Qian, S.; Tan, C. Automated bridge surface crack detection and segmentation using computer vision-based deep learning model. Eng. Appl. Artif. Intell. 2022, 115, 105225. [CrossRef]
56. Tzutalin, D. LabelImg. GitHub Repository. June 2015. Available online: https://pypi.org/project/labelImg/ (accessed on 4 July 2023).